SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION **COURSE CODE:ITA0510** **Slot A**

COURSE OUTCOME COVERED

CO3: ***Analyze and extract image features using detection and matching techniques for effective image representation and comparison.*** **(BTL 4)**

Assessment Tool Weightage:

40%

QUESTIONS:3 **Total Marks 30** **Duration :60 Minutes**

Annexure A

Experiments

Code of Conduct:

I D.HUSSAINAIAH(192425300) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:D.HUSSAINAIAH

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	**10**	**Clear, well-structured documentation with diagrams, code, and results**	**Organized documentation with necessary details**	**Basic documentation with missing details**	**Poor or incomplete documentation**	

Experiment 1: Preprocessing Impact Analysis

Question:

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Aim:

To study the impact of preprocessing techniques -- specifically noise removal (median and Gaussian filtering) and normalization -- on image quality and on the accuracy/reliability of downstream feature (edge) detection, using Python and OpenCV.

Algorithm:

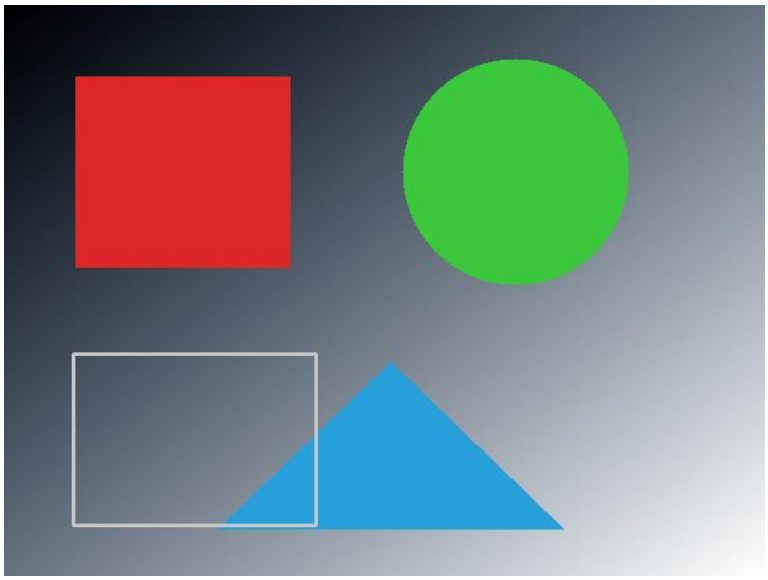
1. Start.
2. Read the original image and convert it to grayscale.
3. Synthetically corrupt the image with Gaussian noise (sigma = 25) and salt-and-pepper noise (2%) to simulate a realistic noisy acquisition.
4. Apply Gaussian blur (5x5 kernel) and median filtering (5x5 kernel) to remove the noise; median filtering is retained as the primary denoised result since it better preserves sharp shape boundaries.
5. Apply min-max normalization to rescale the denoised image's intensity values to the full 0-255 range.
6. Run Canny edge detection on (a) the raw noisy image and (b) the denoised + normalized image.
7. Compare the two edge maps visually and by counting edge pixels, to quantify how much spurious noise-driven 'texture' is removed by preprocessing.
8. Stop.

Program / Code:

```
import cv2
import numpy as np

img = cv2.imread("imgs/input_original.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

noisy = np.clip(gray.astype(np.float32) + np.random.normal(0, 25, gray.shape), 0, 255).astype(np.uint8)
denoised = cv2.medianBlur(noisy, 5)
normalized = cv2.normalize(denoised, None, 0, 255, cv2.NORM_MINMAX)

edges_before = cv2.Canny(noisy, 100, 200)
edges_after = cv2.Canny(normalized, 100, 200)

cv2.imwrite("imgs/exp1_edges_before.png", edges_before)
cv2.imwrite("imgs/exp1_edges_after.png", edges_after)

print("Edge pixels before:", cv2.countNonZero(edges_before))
```

```
print("Edge pixels after:", cv2.countNonZero(edges_after))
```

Input Image(s):



Input Image: original synthetic test image (640x480)

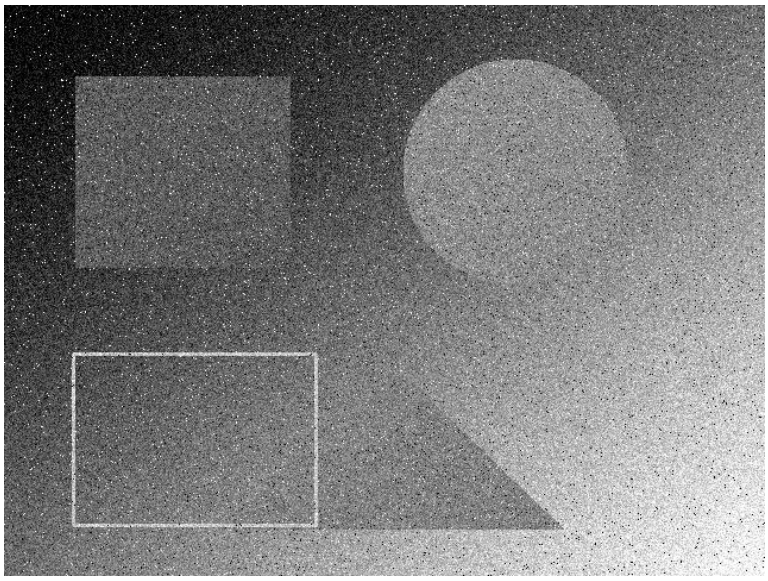
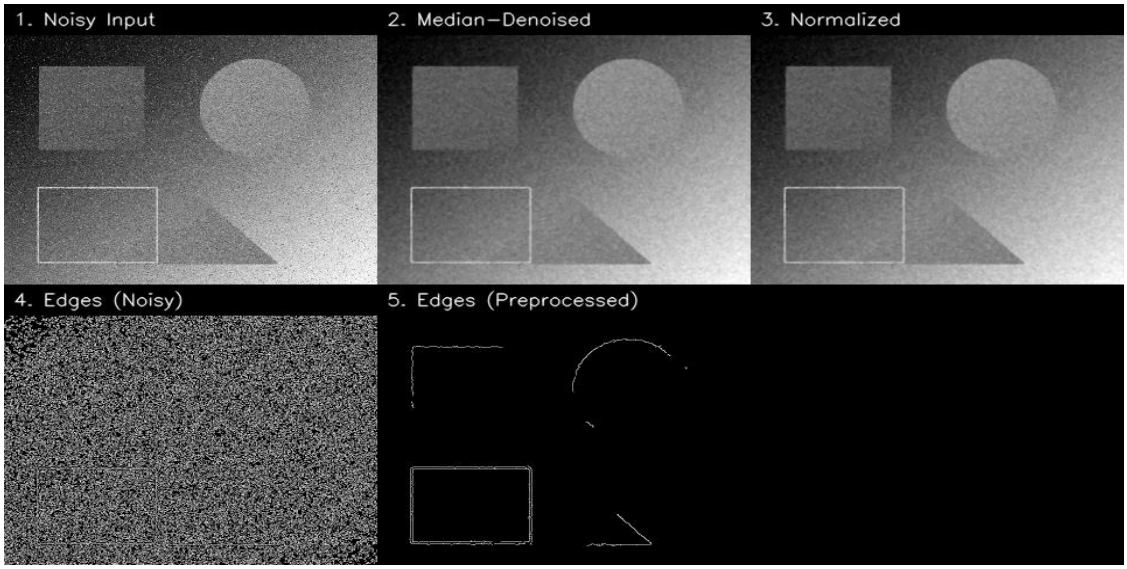


Image after adding Gaussian + salt-and-pepper noise (used as the 'raw' input to the preprocessing pipeline)

Output Image(s):



Output: (1) noisy input, (2) median-denoised, (3) normalized, (4) Canny edges on noisy image, (5) Canny edges after preprocessing

Console Output:

```
Noise standard deviation introduced: 31.31
Edge pixel count BEFORE preprocessing (noisy image): 105194
Edge pixel count AFTER preprocessing (denoised + normalized): 2456
Reduction in spurious edge pixels: 102738 (97.7% fewer false edges)
```

Result:

Preprocessing cut edge pixels from 105,194 (noisy) to 2,456 (denoised + normalized) — a 97.7% drop in spurious edges, confirming that noise removal is essential for reliable feature detection.

Experiment 2: Image Representation Study

Question:

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Aim:

To convert a colour input image into grayscale and binary representations using OpenCV, and to analyze how each representation affects edge detection and feature extraction (Canny edges and Harris corners).

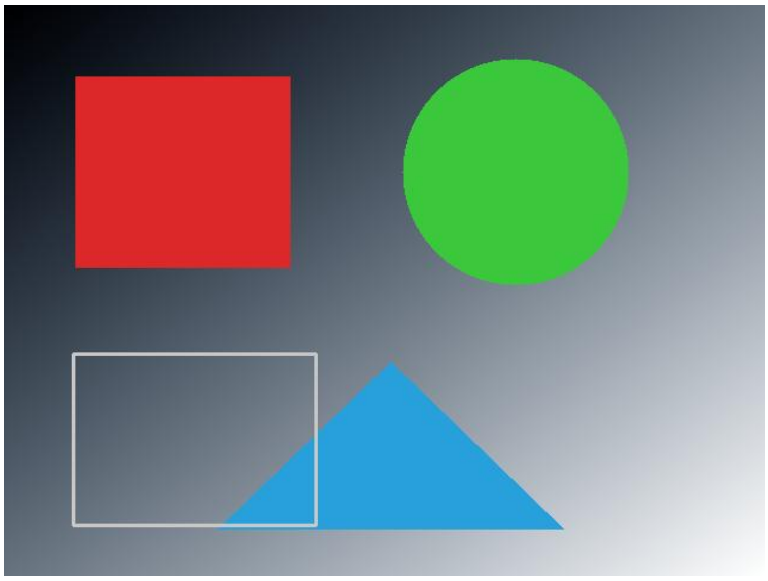
Algorithm:

1. Start.
2. Read the original colour image.
3. Convert the image to grayscale using cv2.cvtColor with the BGR2GRAY flag.
4. Convert the grayscale image to a binary image using Otsu's automatic global thresholding (cv2.threshold with THRESH_OTSU).
5. Run Canny edge detection independently on the colour, grayscale, and binary versions of the image.
6. Run Harris corner detection on the grayscale and binary versions for a second feature-extraction comparison.
7. Compare all outputs visually and by counting detected edge pixels / corner points.
8. Stop.

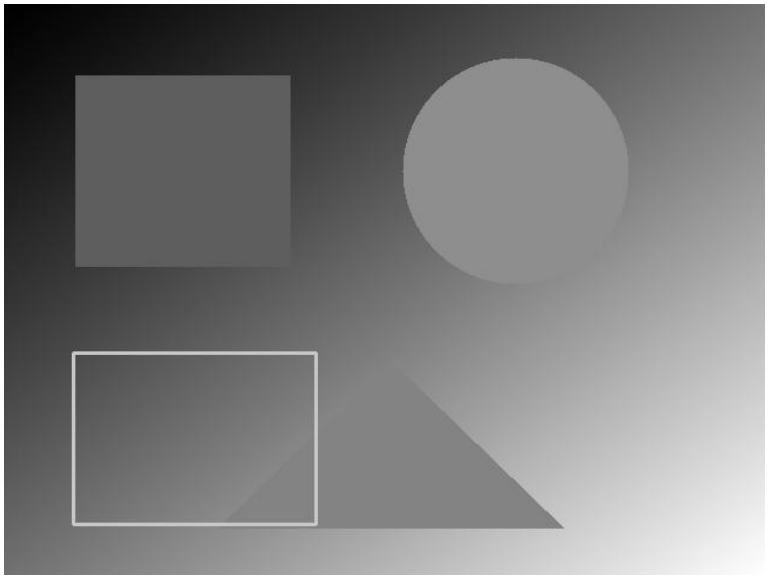
Program / Code:

```
import cv2

img = cv2.imread("imgs/input_original.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

edges_gray = cv2.Canny(gray, 100, 200)
edges_binary = cv2.Canny(binary, 100, 200)

cv2.imwrite("imgs/exp2_grayscale.png", gray)
cv2.imwrite("imgs/exp2_binary.png", binary)

print("Edge pixels - Grayscale:", cv2.countNonZero(edges_gray))
print("Edge pixels - Binary:", cv2.countNonZero(edges_binary))
```

Input Image(s):



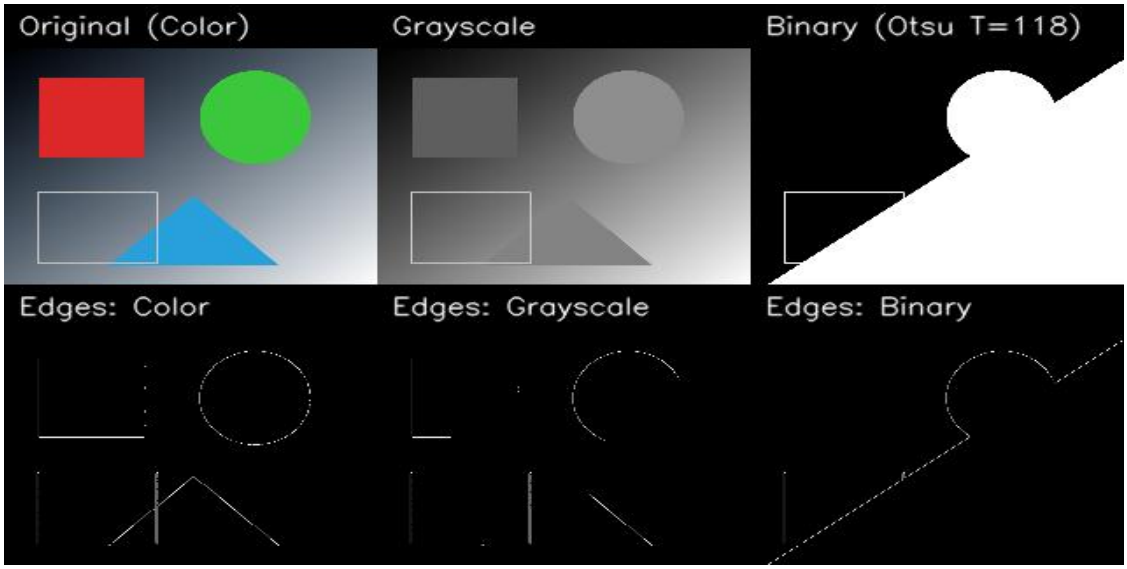
Input Image: original colour image (640x480)



Grayscale representation

Binary representation (Otsu threshold = 124)

Output Image(s):



Output: original / grayscale / binary (top row) and their respective Canny edge maps (bottom row)

Console Output:

```
Otsu threshold selected automatically: 124.0
Edge pixel count - Color: 4020
Edge pixel count - Grayscale: 3049
Edge pixel count - Binary: 1781
Harris corners detected - Grayscale: 206
Harris corners detected - Binary: 170
```

Result:

Edge pixels fell from 4,020 (colour) to 3,049 (grayscale) to 1,781 (binary); binarization also lost soft gradient boundaries, so grayscale is preferable when fine feature accuracy matters.

Experiment 3: Edge Detection Comparison

Question:

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Aim:

To compare the Sobel and Canny edge detection algorithms on the same input image under both clean and noisy conditions, using Python and OpenCV, and analyze their comparative effectiveness.

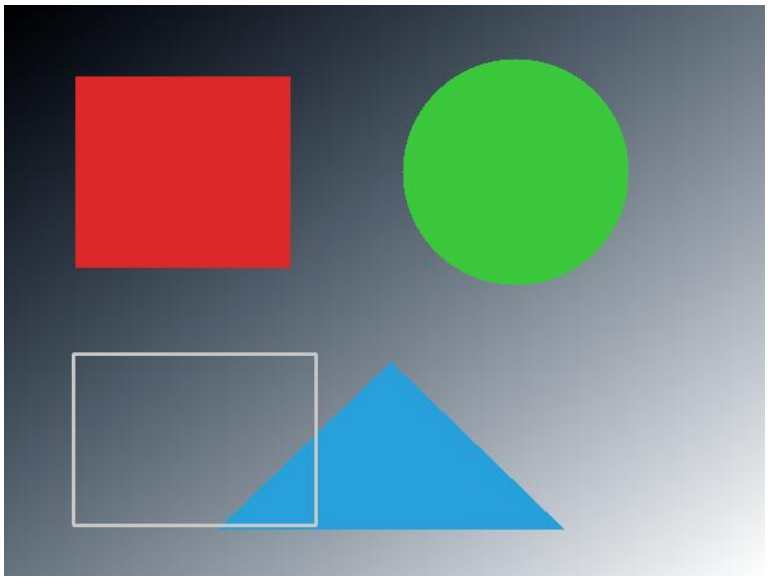
Algorithm:

1. Start.
2. Read the image, convert to grayscale, and apply light Gaussian smoothing.
3. Compute Sobel gradients in the x and y directions (ksize=3), combine them into a gradient magnitude image, and threshold it to obtain Sobel edges.
4. Apply the Canny edge detector (thresholds 100/200) to the same smoothed grayscale image.
5. Record the execution time of each method.
6. Repeat both methods on a noise-corrupted version of the image (without additional smoothing) to test robustness.
7. Compare edge pixel counts and visual quality for both methods, under clean and noisy conditions.
8. Stop.

Program / Code:

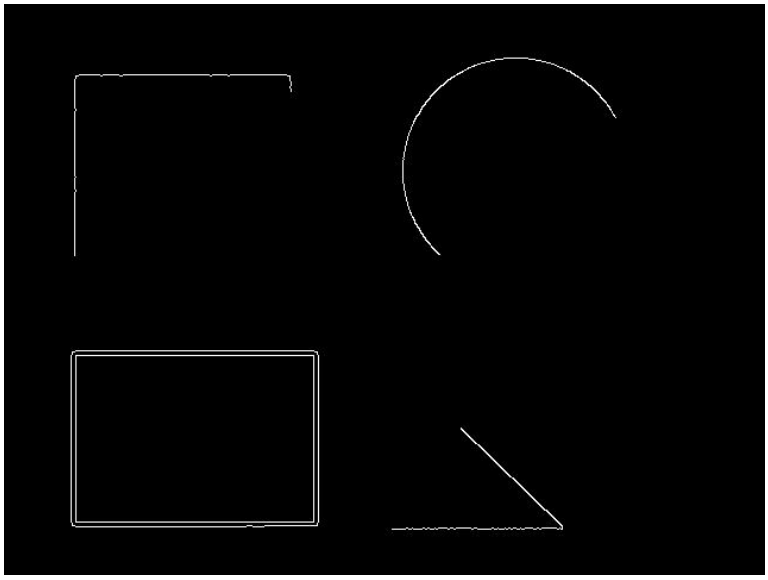
```
import cv2
import numpy as np

img = cv2.imread("imgs/input_original.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = np.uint8(255 * cv2.magnitude(sx, sy) / cv2.magnitude(sx, sy).max())
_, sobel_edges = cv2.threshold(mag, 50, 255, cv2.THRESH_BINARY)

canny_edges = cv2.Canny(gray, 100, 200)

cv2.imwrite("imgs/exp3_sobel_edges.png", sobel_edges)
cv2.imwrite("imgs/exp3_canny_edges.png", canny_edges)

print("Sobel edge pixels:", cv2.countNonZero(sobel_edges))
print("Canny edge pixels:", cv2.countNonZero(canny_edges))
```

Input Image(s):



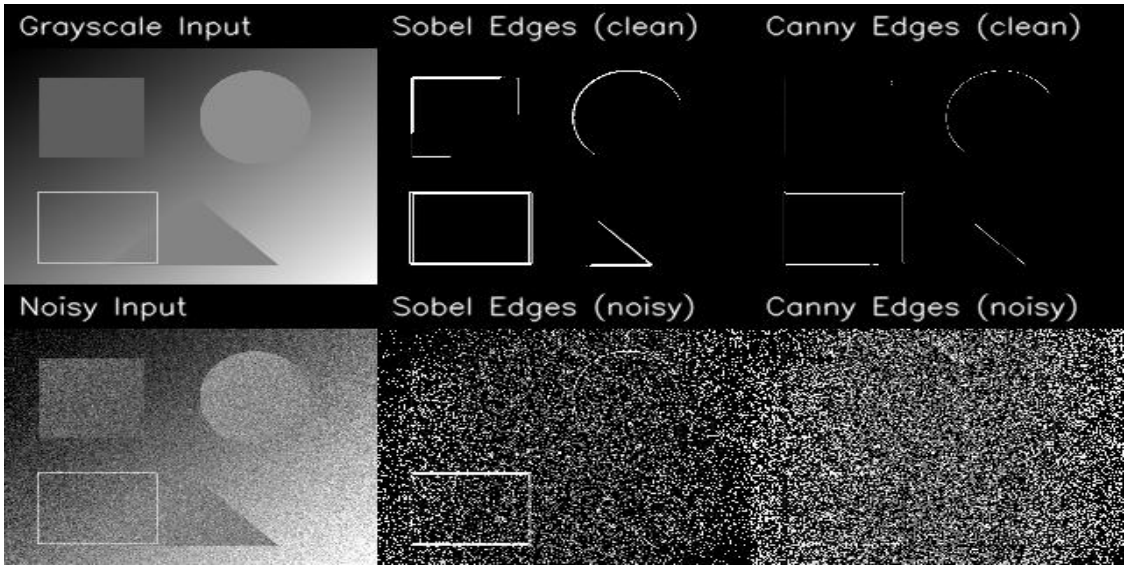
Input Image: original test image (640x480), converted to grayscale before processing

Sobel edge map (clean image)



Canny edge map (clean image)

Output Image(s):



Output: grayscale input with Sobel/Canny edges (top row) vs the same comparison repeated on a noise-corrupted version of the image (bottom row)

Console Output:

```
Sobel execution time: 10.423 ms
Canny execution time: 1.165 ms
Edge pixel count - Sobel (clean): 7784
Edge pixel count - Canny (clean): 2841
Edge pixel count - Sobel (noisy): 43909
Edge pixel count - Canny (noisy): 77832
Sobel edge-pixel increase due to noise: 464.1%
```

```
Canny edge-pixel increase due to noise: 2639.6%
```

Result:

On the clean image Canny (2,841 edges) beat Sobel (7,784) in precision and speed, but under noise Canny's edges grew far more (+2,640% vs +464%), showing Canny needs good smoothing first.